

Sweep Line Algorithm

Assumption on Non-degeneracy

No segment is vertical. // the sweep line always hits a segment at
// a point.

If ≥ 1 vertical segment, imagine all segments are rotated clockwise by a tiny angle and then test for intersection.
This means:

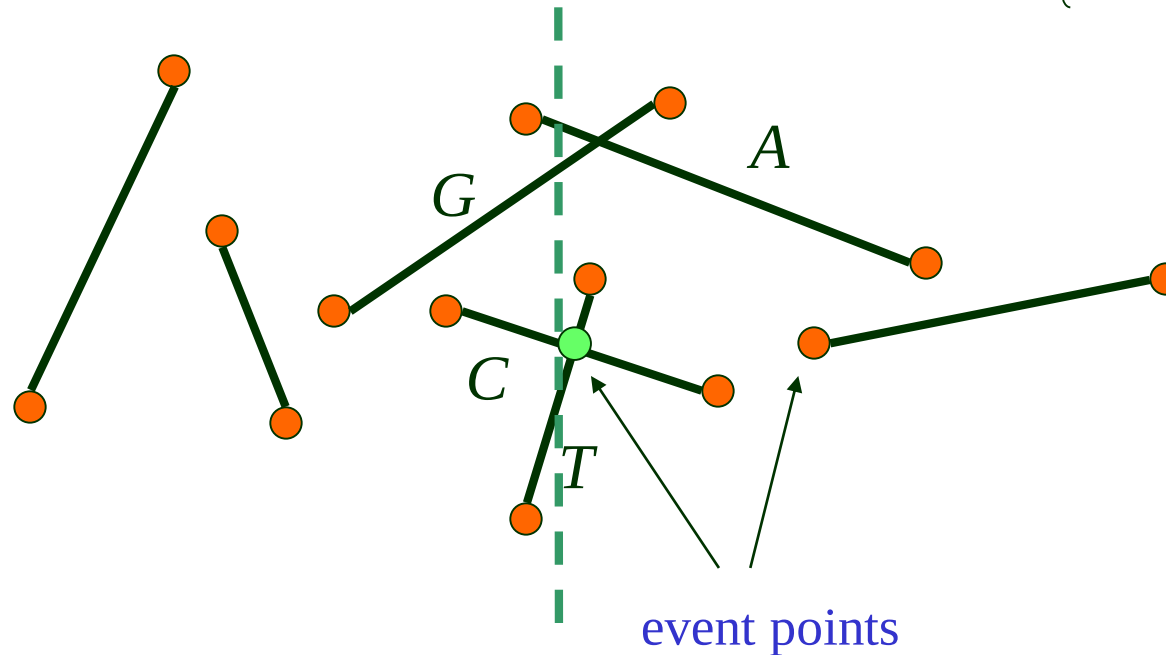
For each vertical segment, we will consider its lower endpoint before upper point.

Sweep Line Status

The set of segments intersecting the sweep line.

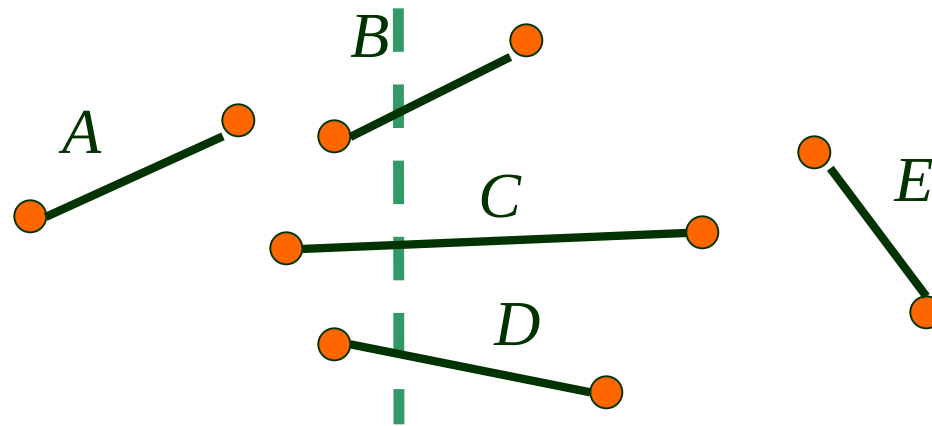
It changes as the sweep line moves, but *not continuously*.

Updates of status happen only at *event points*. { endpoints
intersections



Ordering Segments

A *total order* over the segments that intersect the current position of the sweep line:



$B > C > D$
(A and E not in the ordering)

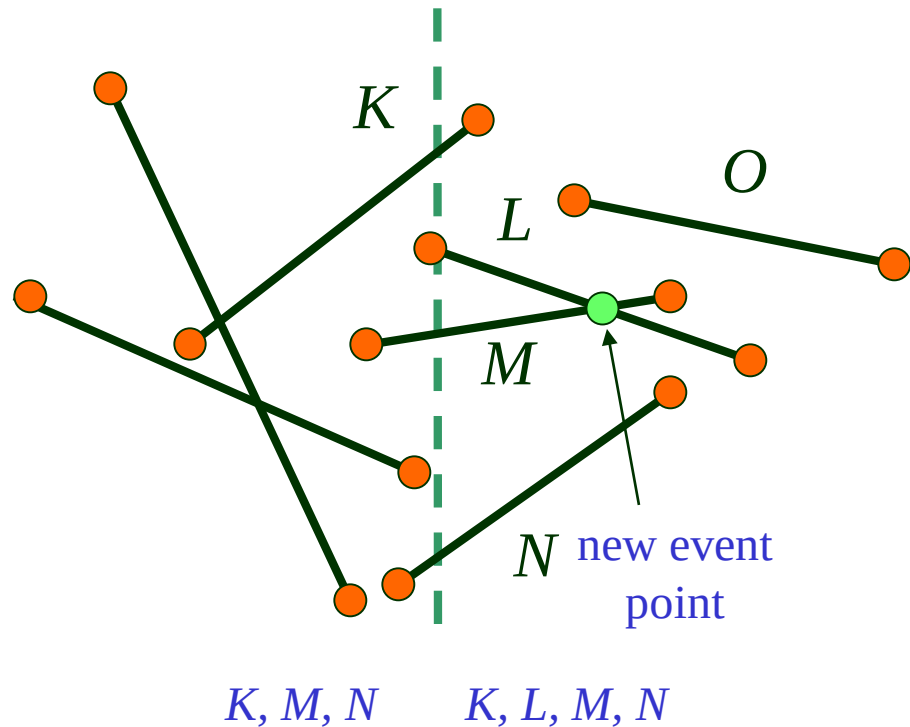
$C > D$
(B drops out of the ordering)

At an event point, the sequence of segments changes:

- ◆ Update the status.
- ◆ Detect the intersections.

Status Update (1)

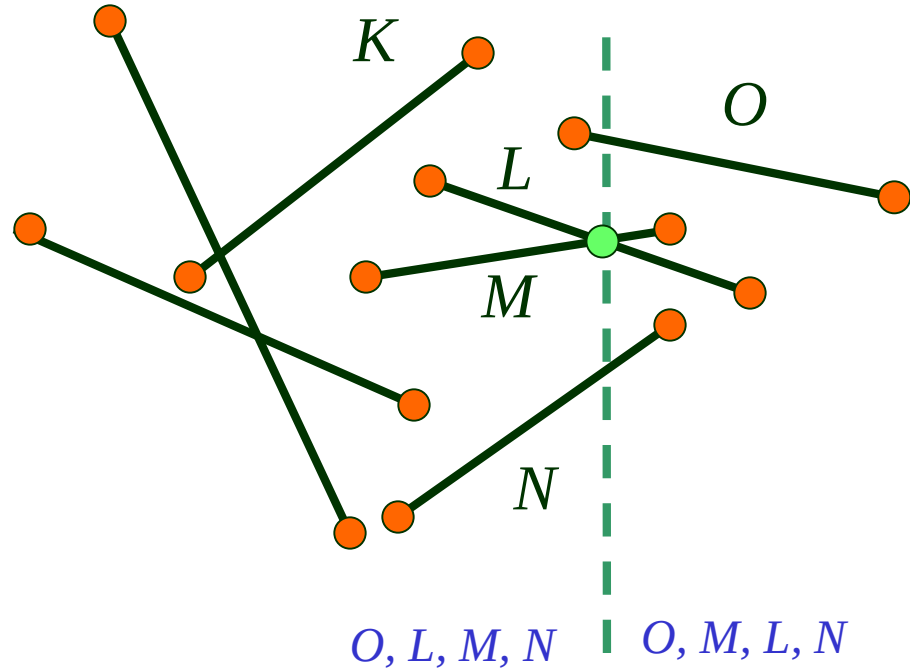
Event point is the left endpoint of a segment.



- ◆ A new segment L intersecting the sweep line
- ◆ Check if L intersects with the segment above (K) and the segment below (M).
- ◆ Intersection(s) are new event points.

Status Update (2)

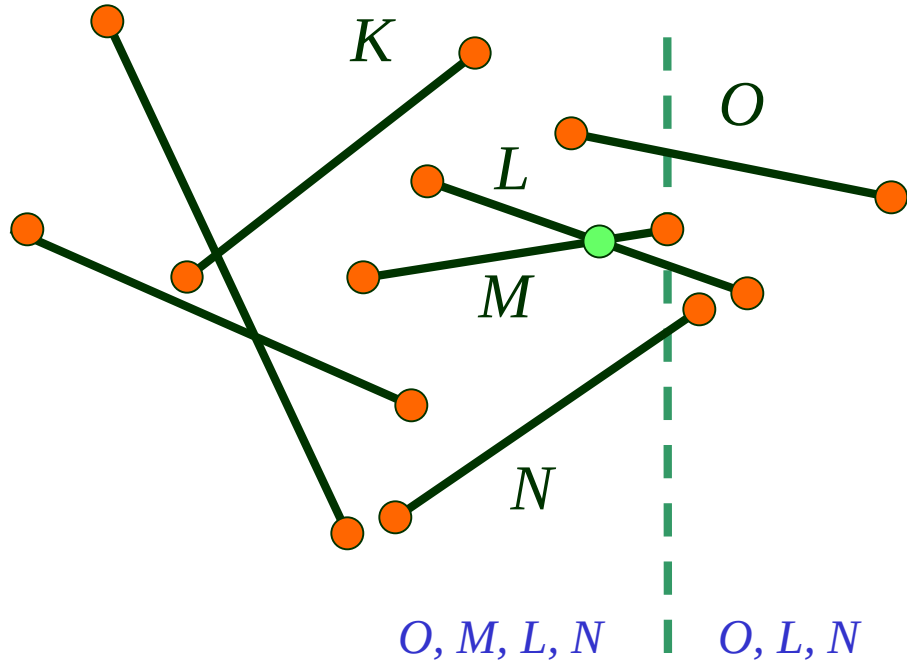
Event point is an intersection.



- ◆ The two intersecting segments (L and M) change order.
- ◆ Check intersection with new neighbors (M with O and L with N).
- ◆ Intersection(s) are new event points.

Status Update (3)

Event point is a lower endpoint of a segment.



- ◆ The two neighbors (O and L) become adjacent.
- ◆ Check if they (O and L) intersect.
- ◆ Intersection is new event point.

Correctness

Invariant holding at any time during the plane sweep.

All intersection points to the left of the sweep line have been computed correctly.

The correctness of the algorithm thus follows.

Data Structure for Event Queue

Ordering of event points:

- ✱ by x -coordinates
- ✱ by y -coordinates in case of a tie in x -coordinates.

Supports the following operations on a segment s .

- fetching the next event // $O(\log m)$
- inserting an event // $O(\log m)$

Every event point p is stored with all segments starting at p .

Data structure: balanced binary search tree (e.g., red-black tree).

$m = \# \text{event points in the queue}$

Data Structure for Sweep-line Status

- ✦ Describes the relationships among the segments intersected by the sweep line.
- ✦ Use a balanced binary search tree T to support the following operations on a segment s .

Insert(T, s)

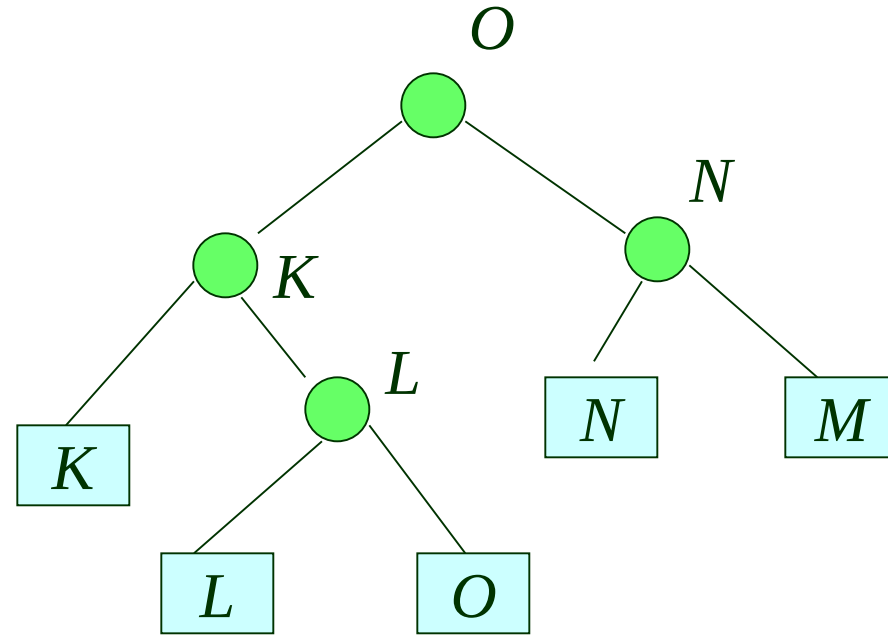
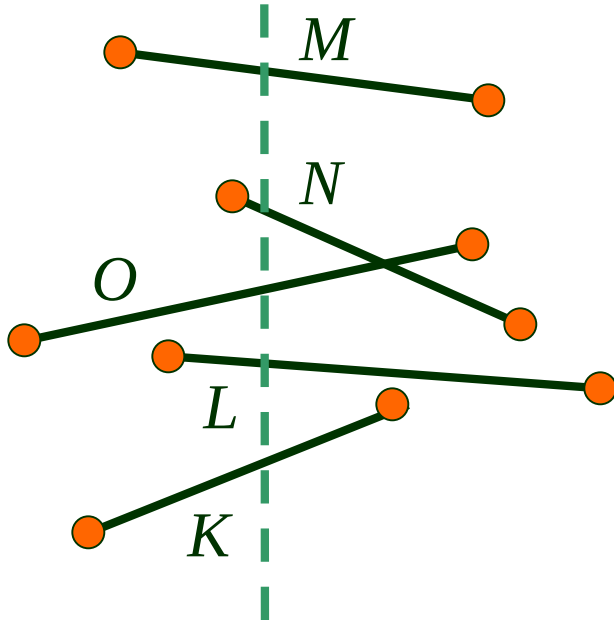
Delete(T, s)

Above(T, s) // segment immediately above s

Below(T, s) // segment immediately below s

- ✦ *e.g.*, Red-black trees, splay trees (key comparisons replaced by cross-product comparisons).
- ✦ $O(\log n)$ for each operation.

An Example (Construction of Tree **T**)



- ◆ The bottom-up order of the segments correspond to the *left-to-right* order of the leaves in the tree **T**.
- ◆ Each internal node stores the segment from the rightmost leaf in its left subtree.

Imagine **n segments** crossing a sweep line at some x-position.

The segments (based on **y-order** from top to bottom) are:

Top

|

| Segment B

| Segment A

| Segment G

| Segment C

|

Bottom

Step 1: Start building the tree

First insert **B**.

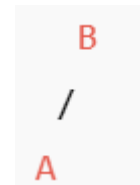
- Tree is empty → **B** becomes the **root**.

Step 2: Insert A

Compare **A** with root **B**:

- Since A is **below** B (lower y-coordinate), A is **left** of B.

Tree now:

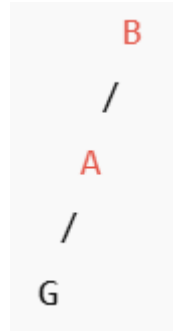


Step 3: Insert G

Compare **G** with root **B**:

- G is **below** B → go left to **A**.
- G is **below** A too → go left again.

Tree now:



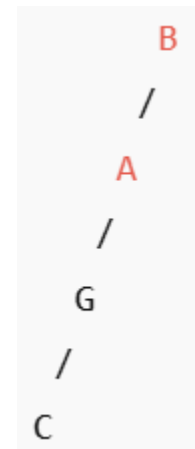
Step 4: Insert C

Compare **C** with **B**:

- C is below B → go left to A. Compare with **A**:
- C is below A → go left to G. Compare with **G**:
- C is **below** G → go left (but G has no left child).

Attach C to left of G.

Final Tree:

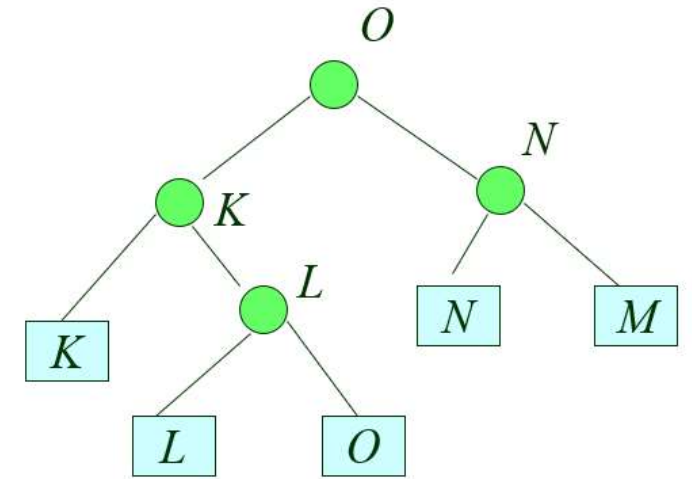
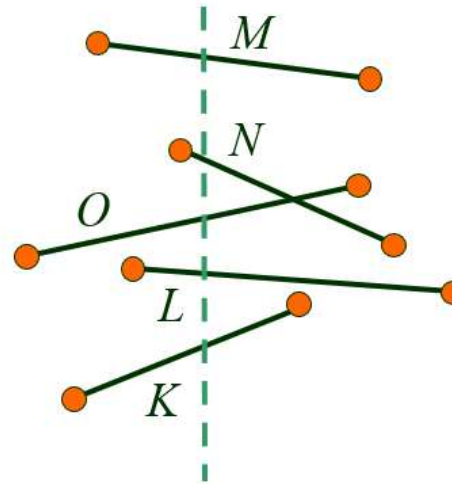


1. Look at the Segments on Sweep Line

At the sweep line (the dotted vertical line), from **bottom to top**, the segments cross in this vertical order:

Bottom to Top Order (low y to high y)

K
L
O
N
M



2. Leaves of the Tree: Bottom-to-Top becomes Left-to-Right

The leaves (bottom-most segments) are placed **left to right** as:

$K \rightarrow L \rightarrow O \rightarrow N \rightarrow M$

These are the leaves at the bottom of the tree!

(Notice: **Bottom in figure** $\rightarrow$ **Left in tree**)

3. Build Tree by Grouping

✓ Now **pair up segments** left-to-right:

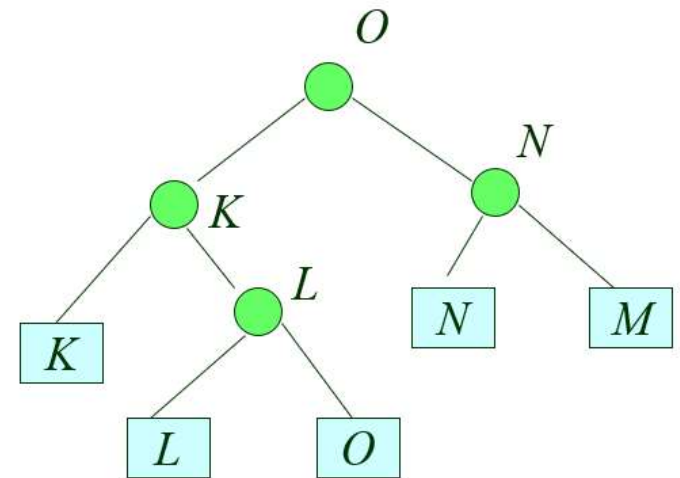
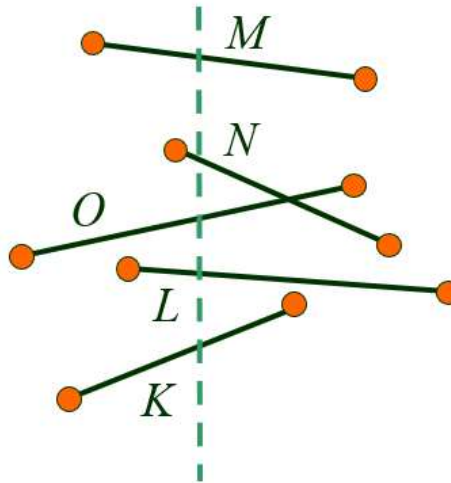
- **K and L** become a small subtree
- **O alone** (wait)
- **N and M** become a small subtree

4. Build Small Subtrees First

- Create a parent node for **K and L** → label this parent by the **rightmost segment in its left child**, so here label is **K**.
- Create a parent node for **N and M** → label this parent by **N**.



- O is still alone.

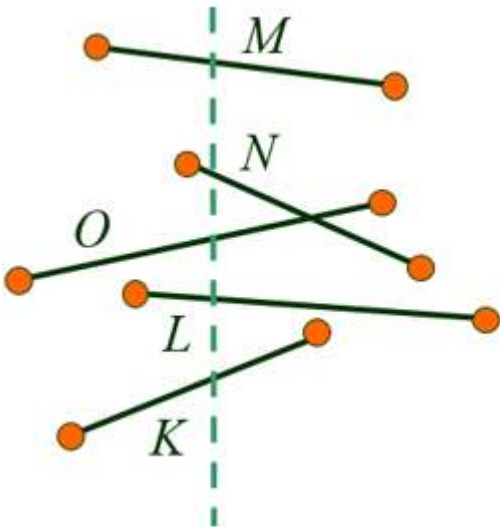


5. Build the Final Tree

Now combine:

- Combine **(K)** and **(O)** first: create a parent node for (K and O). Label with **K** again (rightmost in left subtree is still K).
- Then combine this new node with **(N)**.

The final tree root label is **O**, because **rightmost in left subtree** at final combination is **O**.



Final Tree Structure:

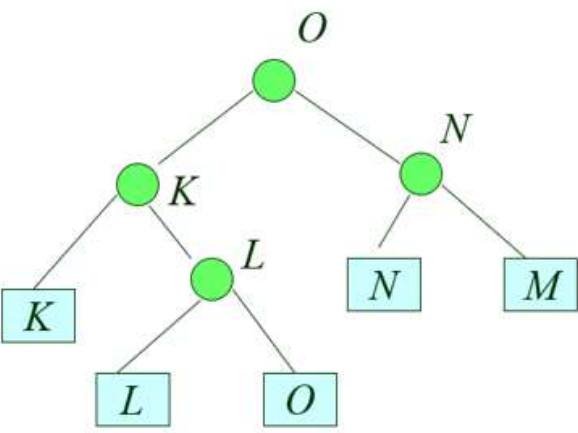


But neatly drawn, it matches your diagram:

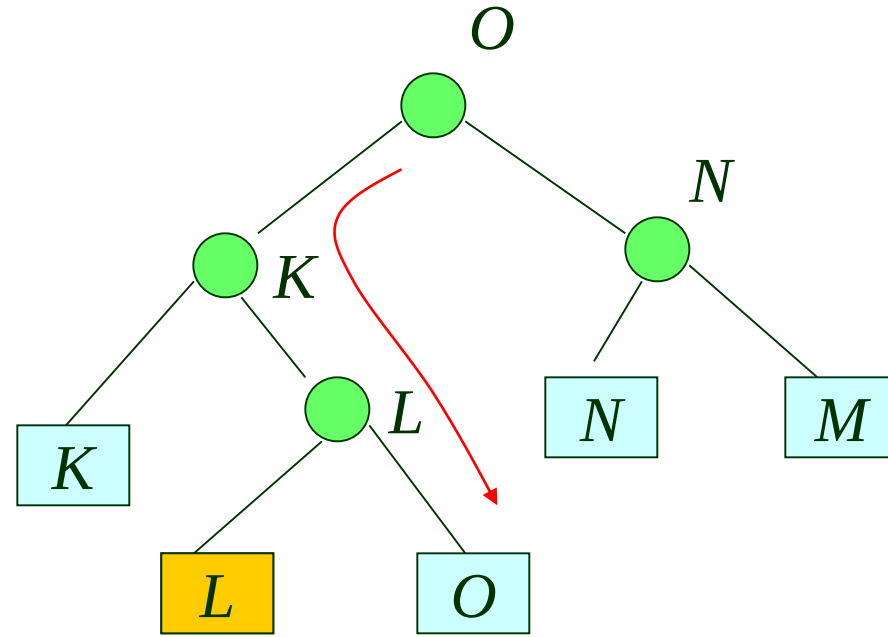
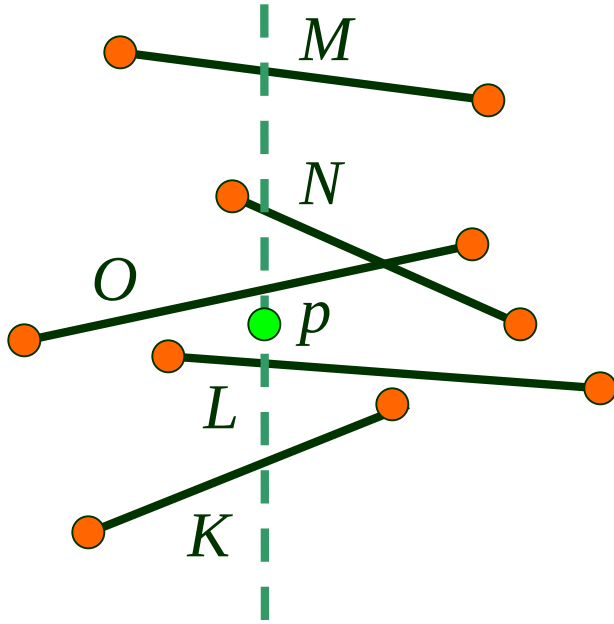
- K and L grouped under K
- K and O grouped under O
- N and M grouped under N
- O is root

Two Key Rules:

- **Bottom-to-Top order** → **Left-to-Right order of leaves.**
- **Each internal node label = segment from rightmost leaf in left subtree.**



Additional Operation



Searching for the segment immediately below some point p on the sweep line.

- Descend binary search tree all the way down to a leaf.
- Outputs either this leaf or the leaf immediately to its left .

$O(\log n)$ time

The Algorithm

FindIntersections(S)

Input: a set S of line segments

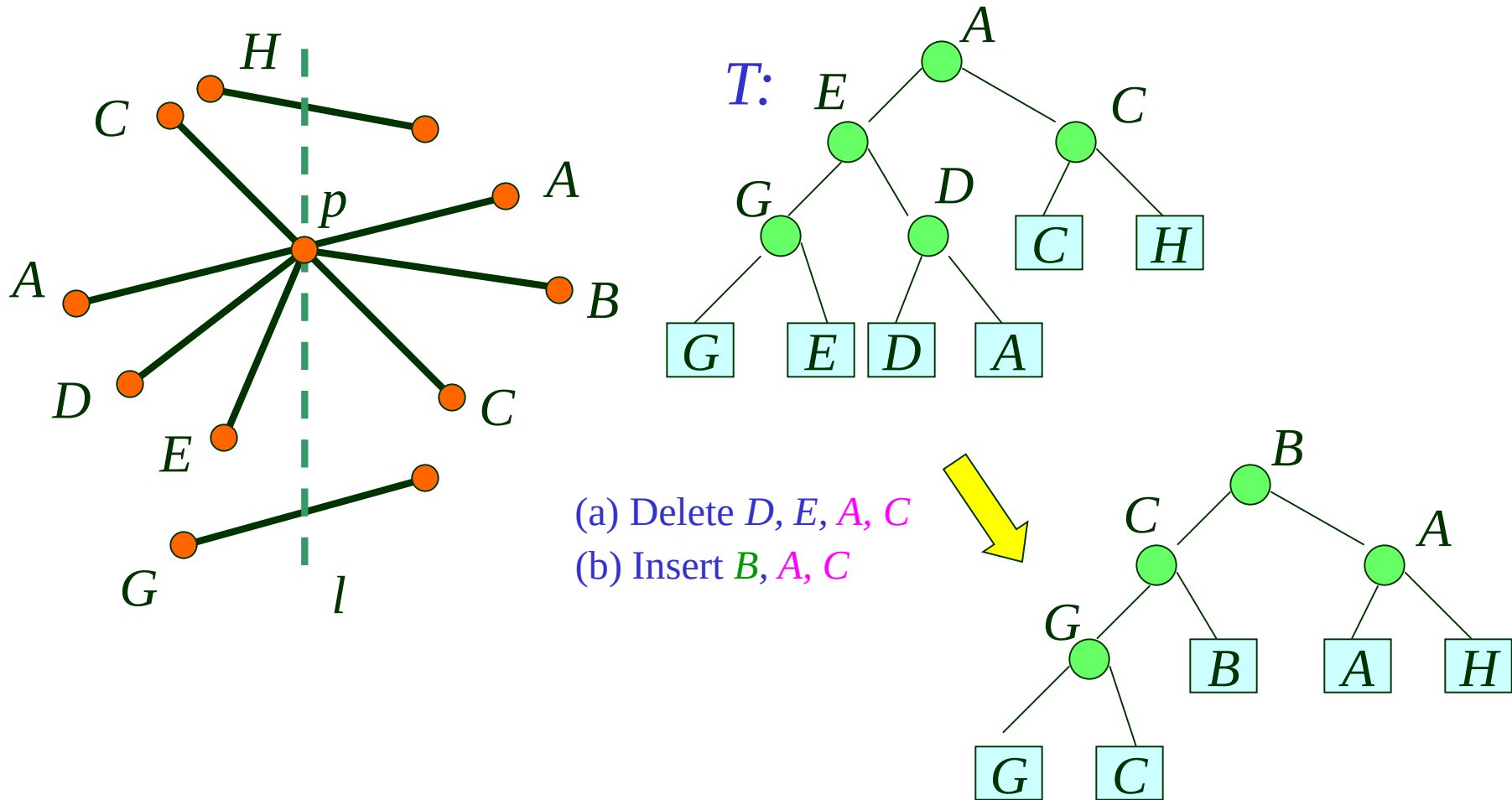
Output: all intersection points and for each intersection the segment containing it.

1. $Q \leftarrow \emptyset$ // initialize an empty event queue
2. Insert the segment endpoints into Q // store with every left endpoint
// the corresponding segments
3. $T \leftarrow \emptyset$ // initialize an empty status structure
4. **while** $Q \neq \emptyset$
5. **do** extract the next event point p
6. $Q \leftarrow Q - \{p\}$
7. HandleEventPoint(p)

Handling Event Points

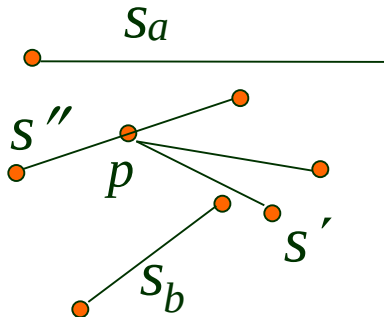
Status updates (1) – (3) presented earlier.

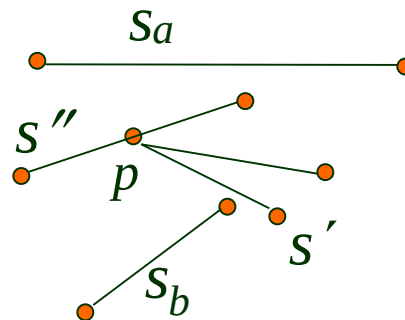
Degeneracy: several segments are involved in one event point (tricky).



The Code of Event Handling

HandleEventPoint(p)

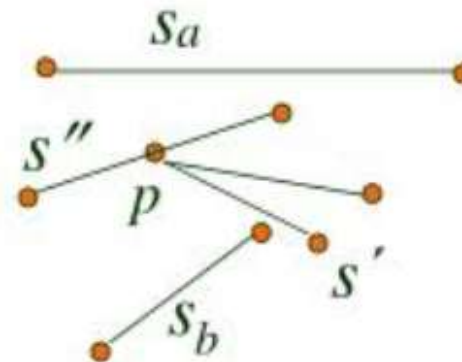
1. $L(p) \leftarrow \{ \text{segments with left endpoint } p \}$ // they are stored with p
 2. $S(p) \leftarrow \{ \text{segments containing } p \}$ // all adjacent in T and found by a search
 3. $R(p) = \{ \text{segments with right endpoint } p \}$ // $L(p) \subseteq S(p)$
 4. $C(p) = \{ \text{segments with } p \text{ in interior} \}$ // $C(p) \subseteq S(p)$
 5. if $|L \cup R \cup C| > 1$
 6. then report p as an intersection along with L, R, C
 7. $T \leftarrow T - (R \cup C)$
 8. $T \leftarrow T \cup (L \cup C)$ // order as intersected by sweep line just to the
 // right of p . segments in $C(p)$ have order reversed.
 9. if $L \cup C = \emptyset$ // right endpoint only
 10. then let s_b and s_a be the neighbors right below and above p in T
 11. FindNewEvent(s_b, s_a, p)
 12. else $s' \leftarrow$ lowest segment in $L \cup C$
 13. $s_b \leftarrow$ segment right below s'
 14. FindNewEvent(s_b, s', p)
 15. $s'' \leftarrow$ highest segment in $L \cup C$
 16. $s_a \leftarrow$ segment right above s''
 17. FindNewEvent(s'', s_a, p)
- 



PTO (The same Algor – Image form)

HandleEventPoint(p)

1. $L(p) \leftarrow \{ \text{segments with left endpoint } p \}$ // they are stored with p
2. $S(p) \leftarrow \{ \text{segments containing } p \}$ // all adjacent in T and found by a search
3. $R(p) = \{ \text{segments with right endpoint } p \}$ // $L(p) \subseteq S(p)$
4. $C(p) = \{ \text{segments with } p \text{ in interior} \}$ // $C(p) \subseteq S(p)$
5. if $|L \cup R \cup C| > 1$
6. then report p as an intersection along with L, R, C
7. $T \leftarrow T - (R \cup C)$
8. $T \leftarrow T \cup (L \cup C)$ // order as intersected by sweep line just to the
// right of p . segments in $C(p)$ have order reversed.
9. if $L \cup C = \emptyset$ // right endpoint only
10. then let s_b and s_a be the neighbors right below and above p in T
11. FindNewEvent(s_b, s_a, p)
12. else $s' \leftarrow$ lowest segment in $L \cup C$
13. $s_b \leftarrow$ segment right below s'
14. FindNewEvent(s_b, s', p)
15. $s'' \leftarrow$ highest segment in $L \cup C$
16. $s_a \leftarrow$ segment right above s''
17. FindNewEvent(s'', s_a, p)



Not needed, bcoz this slide is same as previous slide

Finding New Event

FindNewEvent(s_l, s_r, p)

1. **if** s_l and s_r intersect to the right of p // sweep line position
2. **then** insert the intersection point as an event in Q

Correctness

Lemma Algorithm `FindIntersections` computes all intersections and their containing segments correctly.

Proof By induction. Let p be an intersection point and assume all intersections with a higher priority have been computed correctly.

★ p is an endpoint. $\Rightarrow$ stored in the event queue Q .
 $L(p)$ is also stored in Q .
 $R(p)$ and $C(p)$ are stored in T and will be found. } All involved are determined correctly.

★ p is not an endpoint. We show that p will be inserted into Q .

All involved segments have p in interior. Order them by polar angle.
Let A and B be neighboring segments.

$\Rightarrow$ There exists event point $q < p$ after which A and B become adjacent..

$\Rightarrow$ By induction, q was handled correctly and p is detected and stored in Q

